

QUBO: A Reference for Formulation, Validation and Tuning

Contents

0. Notation and conventions	1
1. Algebraic foundation	2
2. Translating the objective function	3
3. Catalogue of penalty terms	5
4. Construction of slack variables	10
5. Choice of the penalty weight	11
6. Validation suite	13
7. Precision criteria	13
8. Diagnostic table	14
9. Comparing solvers	14
10. Common errors	15
11. Limits of applicability of QUBO	15

0. Notation and conventions

The QUBO (Quadratic Unconstrained Binary Optimization) problem:

$$\min_{x \in \{0,1\}^n} x^T Q x$$

where $Q \in \mathbb{R}^{n \times n}$ is upper triangular. For $i < j$, the entry Q_{ij} is the coefficient of the term $x_i x_j$; the entry Q_{ii} is the coefficient of the linear term x_i .

A symmetric matrix S is converted to upper triangular form by $Q = \text{triu}(S) + \text{triu}(S, 1)$, that is, by doubling the off-diagonal entries. The two conventions **must not be mixed** across functions in the same codebase; energies would differ by a factor of two on the quadratic terms.

An additive constant c in the objective does not affect the argument of the minimum and is not written into Q . It is carried separately and added back only when comparing energies against objective function values.

0.1 Glossary

Terms that recur throughout, in plain form. Each is treated formally in the section named.

Binary variable: A variable taking only the values 0 and 1, read as a yes/no decision: item taken or not, vertex coloured or not.

Objective function f : The quantity to be minimized: cost, distance, risk. §2.

Constraint: A condition a solution must meet to count as valid, such as a capacity limit. QUBO has no mechanism for constraints, which is why they are converted into penalties.

Feasible solution: One that satisfies every constraint. The set of these is written F .

Penalty term P : An expression added to the objective that is zero on feasible solutions and positive elsewhere, so that violating a constraint raises the energy. It stands in for a constraint that QUBO cannot express directly. §1.2, §3.

Penalty weight λ : The multiplier on a penalty term, setting how expensive a violation is relative to the objective. Too small and violations pay for themselves; too large and the objective stops influencing the result. §5.

Slack variable: An extra variable carrying no meaning in the original problem, added to turn an inequality into an equality: “spend at most 11” becomes “spend exactly 11, where part of the 11 may go unspent”. The conversion is needed because only an equality can be squared into a penalty. §4.

Quadratization: Reduction of a term involving three or more variables to terms involving at most two, by giving the product of two of them a new name and forcing that name to hold. QUBO admits pairwise terms only. §3.10.

Energy: The value $x^T Q x$ of a given assignment. The solver searches for the assignment of least energy.

Offset: A constant added to every energy alike. It cannot change which assignment is best and is therefore kept outside Q . §0.

Encoding: The representation of a non-binary variable, integer or categorical, by a group of binary variables. §2.3.

Embedding: The mapping of the formulation onto the physical qubits of a device whose connectivity is limited. A variable with more neighbours than the hardware degree allows becomes a chain of qubits constrained to agree. §3.7.

Sweep: A systematic scan over values of a parameter, here λ , measuring result quality at each value. §5.4.

1. Algebraic foundation

1.1 Idempotence of binary variables

For $x \in \{0, 1\}$,

$$x \in \{0, 1\} \iff x(x - 1) = 0 \iff x^2 = x.$$

This equivalence is the sole point at which the binarity of the variables enters the formulation, and it is the source of three consequences that determine the structure of Q :

Consequence A (collapse of degrees): Every polynomial over $\{0, 1\}^n$ is equivalent to a multilinear polynomial, that is, one in which no variable appears in a power higher than the first. Powers x_i^k for $k \geq 1$ reduce to x_i .

Consequence B (placement of linear terms): Since $x_i^2 = x_i$, the linear term ax_i is identical to the quadratic term ax_i^2 . In the form $x^T Q x$, linear coefficients therefore belong on the diagonal. The absence of a separate linear vector is not a notational omission but a consequence of idempotence.

Consequence C (expansion of the square): For arbitrary $a \in \mathbb{R}^n$, $b \in \mathbb{R}$:

$$\left(\sum_i a_i x_i - b \right)^2 = \sum_{i < j} 2a_i a_j x_i x_j + \sum_i a_i (a_i - 2b) x_i + b^2$$

Derivation: expanding the square gives $\sum_{i,j} a_i a_j x_i x_j - 2b \sum_i a_i x_i + b^2$; the terms with $i = j$ yield $a_i^2 x_i^2 = a_i^2 x_i$ by Consequence A and move to the linear part, while the terms with $i \neq j$ pair up into $i < j$ with a factor of two.

This expansion is the core of the conversion of every equality constraint and is used repeatedly in §3.

1.2 Requirements on a penalty term

A constraint is introduced into the objective through a term $\lambda P(x)$, where P must satisfy:

1. $P(x) \geq 0$ for all $x \in \{0, 1\}^n$,
2. $P(x) = 0$ if and only if x satisfies the constraint,
3. P is at most quadratic in x (otherwise quadratization is required; see §3.10).

The quantity

$$\Delta P_{\min} = \min\{P(x) : x \in \{0, 1\}^n, P(x) > 0\}$$

is the smallest nonzero value of the penalty. It enters the lower bound on λ in §5 and is stated for every pattern in §3.

2. Translating the objective function

The catalogue in §3 covers constraints. This section covers the other half of the translation: turning the objective of the original problem into a quadratic form over binary variables. The steps are ordered so that each one produces the input required by the next.

2.1 The procedure

1. **Fix the direction of optimization** (§2.2). QUBO minimizes; a maximization must be negated.
2. **Encode every non-binary decision variable** into binary variables (§2.3). Integer, categorical and discretized continuous variables all require an explicit encoding, and the choice of encoding constrains everything downstream.
3. **Express the objective as a polynomial** in the encoded variables and sort its terms by degree (§2.4).
4. **Reduce the degree to at most two** by the substitution of §3.10, if any term of degree three or higher remains.
5. **Translate each constraint** into a penalty term (§3), introducing slack variables where the constraint is an inequality (§4).
6. **Normalize the objective and choose the weights λ** (§2.6, §5).
7. **Assemble Q** : quadratic coefficients off the diagonal, linear coefficients on the diagonal by Consequence B, constants into the separate offset (§0).
8. **Validate** (§6).

Steps 1 to 4 concern the objective and are the subject of this section. Step 2 is the one that most often decides whether the resulting formulation is usable, because the encoding fixes both the variable count and the spread of coefficient magnitudes.

2.2 Direction of optimization

QUBO is defined as a minimization. A maximization problem is converted by negating the objective:

$$\max_x f(x) \longrightarrow \min_x (-f(x))$$

The negation applies to the objective only. Penalty terms are always added with a positive weight, since their role, non-negativity vanishing exactly on the feasible set, is independent of the direction of optimization. Negating the whole assembled expression, penalties included, inverts their meaning and rewards infeasibility.

2.3 Encoding non-binary variables

An integer variable $z \in \{0, 1, \dots, M\}$ must be represented by binary variables. The usual options are the following.

Encoding	Variables	Constraint required	Coefficient spread
Binary (logarithmic)	$\lceil \log_2(M+1) \rceil$	none, with truncated coefficients (§4)	$1 : M$
One-hot	$M + 1$	§3.1 per variable	uniform
Unary (thermometer)	M	§3.6 per adjacent pair	uniform
Domain wall	M	§3.6 per adjacent pair	uniform

Binary encoding: With $z = \sum_k c_k s_k$ and the truncated coefficients of §4, this uses the fewest variables and needs no penalty term. Its cost is that the coefficients span a range of $1 : M$, and after squaring in the objective a range of $1 : M^2$. This enters $\max |Q|$ directly and worsens the precision requirement β of §7.2. The encoding that looks cheapest by variable count is therefore often the one that fails on hardware.

One-hot encoding: Setting $z = \sum_{k=0}^M k x_k$ with the constraint of §3.1 keeps all coefficients uniform, at the price of $M + 1$ variables and a complete coupling graph per encoded variable.

Unary encoding: Setting $z = \sum_k x_k$ with an ordering constraint $x_{k+1} \leq x_k$ from §3.6 makes the ones form a prefix. Its advantage is locality: a single bit flip satisfies $|\Delta z| = 1$, which suits local-move heuristics. Under binary encoding a transition between adjacent values of z may require up to $\lceil \log_2(M+1) \rceil$ simultaneous flips, as in $7 \rightarrow 8$ with $0111 \rightarrow 1000$, and single-flip search can stall between them.

Categorical variables: With M unordered categories there is no meaningful arithmetic encoding, so one-hot with §3.1 is used.

Continuous variables: These must be discretized onto a grid before any of the above applies. The discretization error is a modelling decision that no later step recovers; a solution exact with respect to the grid may still be far from the continuous optimum.

2.4 Objective terms by degree

Once all variables are binary, the objective is a polynomial and its terms are placed according to degree:

Degree	Term	Destination
0	c	separate offset (§0), not written into Q
1	$c_i x_i$	$Q_{ii} += c_i$ by Consequence B
2	$c_{ij} x_i x_j$	$Q_{ij} += c_{ij}$ for $i < j$ (§0 on convention)
≥ 3	$c x_i x_j x_k \dots$	quadratization by §3.10

Examples: The following terms arise in standard problems.

Origin	Term	Deg.	Placement in Q
Constant left by expanding a penalty (§3.9)	λb^2	0	offset only
Value of item i in the knapsack, after negation (§2.2)	$-v_i x_i$	1	$Q_{ii} -= v_i$
Fixed cost of opening facility i	$f_i y_i$	1	$Q_{ii} += f_i$
Covariance of assets i, j in portfolio risk	$\sigma_{ij} x_i x_j$	2	$Q_{ij} += \sigma_{ij}$
Edge $\{i, j\}$ in max cut, after negation	$-w_{ij}(x_i + x_j - 2x_i x_j)$	2	$Q_{ii}, Q_{jj} -= w_{ij};$ $Q_{ij} += 2w_{ij}$
Leg of a tour between positions p and $p+1$	$d_{ij} x_{i,p} x_{j,p+1}$	2	entry of the two one-hot bits
Cost incurred only if three choices coincide	$c x_i x_j x_k$	3	§3.10, one auxiliary variable

Shared algebra of objective and penalty terms: Writing $P_{3,8}$ for the penalty of §3.8, the max cut term satisfies

$$-w_{ij}(x_i + x_j - 2x_i x_j) = w_{ij}P_{3.8} - w_{ij}$$

Max cut is therefore the problem of satisfying $x_i \neq x_j$ on every edge as far as possible, with weight w_{ij} . The same expression serves as an objective term and as a penalty; what distinguishes the two is the role assigned to it, not the algebraic form.

Degree under encoding: A polynomial of degree d in the original integer variables remains of degree d in the binary variables, for every encoding in §2.3. A bilinear objective in integers therefore stays quadratic and needs no quadratization; a cubic one needs it regardless of the encoding chosen.

Degree under squaring: A term z^2 with $z = \sum_k c_k s_k$ expands to $\sum_{k,l} c_k c_l s_k s_l$, which is of degree two, so a quadratic objective in integer variables requires no quadratization. The cost is not removed but relocated: the number of quadratic terms per squared variable grows to $\binom{m}{2}$, where m is the number of bits of the encoding, and the products $c_k c_l$ span the square of the original coefficient range, which enters $\max |Q|$ and hence §7.2.

2.5 Objectives outside the quadratic class

QUBO expresses exactly the quadratic polynomials over binary variables. An objective outside this class must either be reformulated into it or handled by an outer loop.

Deviation from a target: A requirement that $g(x)$ approach a value b is expressed as $(g(x) - b)^2$, which is quadratic for linear g and is the natural form in QUBO. The number partitioning problem consists of nothing else.

Absolute value: $|g(x)|$ has no direct quadratic form. When the intent is proximity to a target, replace it with the square above, accepting that large deviations are penalized more heavily than proportionally.

Minimax: Minimizing $\max_i g_i(x)$ requires an auxiliary variable bounded from below by every g_i , hence one inequality per term, each with its own slack by §3.9. The cost usually exceeds the benefit; a weighted sum of squares is the common substitute, and it does change the problem being solved.

Ratios: An objective $f(x)/g(x)$ is not expressible. Dinkelbach's parametrization replaces it with a sequence of QUBOs $\min_x (f(x) - t g(x))$, updating t from the solution of the previous one until the value converges.

Each of these substitutions changes the problem. Whether the change is acceptable is a modelling question, and the substitution should be recorded so that later comparisons against the exact solver in §9 are made against the same problem.

2.6 Commensurability of objective and penalties

The objective and the penalty terms are summed into a single energy, so only their relative scale matters. Before choosing λ , determine

$$\Delta f = \max_{x \in F} f(x) - \min_{x \in F} f(x)$$

over the feasible set F , or a bound on it. This is the quantity that governs the lower bound of §5.1, and normalizing the objective so that $\Delta f \approx 1$ makes the values of λ interpretable and comparable across instances.

Rescaling the objective by a factor α requires rescaling every λ by the same α . Energies obtained under different scalings are not comparable, which is the same caution as for different values of λ in §10.

3. Catalogue of penalty terms

3.0a Algebraic summary

§	Constraint	Penalty term P	$\Delta P_{\min}$	Aux.
3.1	$\sum_i x_i = 1$	$(\sum_i x_i - 1)^2$	1	0

§	Constraint	Penalty term P	$\Delta P_{\min}$	Aux.
3.2	$\sum_i x_i = k$	$(\sum_i x_i - k)^2$	1	0
3.3	$\sum_i x_i \leq 1$	$\sum_{i < j} x_i x_j$	1	0
3.4	$x_i + x_j \leq 1$	$x_i x_j$	1	0
3.5	$x_i + x_j \geq 1$	$1 - x_i - x_j + x_i x_j$	1	0
3.6	$x_i \leq x_j$	$x_i - x_i x_j$	1	0
3.7	$x_i = x_j$	$x_i + x_j - 2x_i x_j$	1	0
3.8	$x_i \neq x_j$	$1 - x_i - x_j + 2x_i x_j$	1	0
3.9	$\sum_i a_i x_i \leq b$	$(\sum_i a_i x_i + \sum_k c_k s_k - b)^2$	depends on a	$\lceil \log_2(b+1) \rceil$
3.10	$y = x_1 x_2$	$3y + x_1 x_2 - 2x_1 y - 2x_2 y$	1	1

3.0b Selection guide

§	Pattern	Situation in which it arises
3.1	Exactly one of n	encoding a categorical choice: one machine per job, one colour per vertex, one city per tour position
3.2	Exactly k of n	cardinality constraint: portfolio of fixed size, selection of k centres or facilities
3.3	At most one of n	optional exclusive choice: a resource that may stay idle, a slot that need not be filled
3.4	Pair exclusion	conflict edge: independent set, overlapping time intervals, incompatible components
3.5	Pair coverage	covering edge: vertex cover, every edge must be served by an endpoint
3.6	Implication	precedence and prerequisites: a task may start only if its predecessor ran, activating a feature requires its parent
3.7	Pair agreement	chains in minor embedding, aliasing of a variable duplicated across subproblems
3.8	Pair disagreement	bipartition: max cut, splitting a set into two halves, parity conditions
3.9	Linear inequality	capacity and budget limits: knapsack, bin packing, resource ceilings, minimum service levels
3.10	Product quadratization	reduction of a higher-order objective (PUBO) to quadratic form

3.0c Inverse index: problem to patterns

Problem	Patterns used
Knapsack	3.9
Bin packing	3.1 per item, 3.9 per bin
Graph colouring	3.1 per vertex, 3.4 per edge
Maximum independent set	3.4 per edge
Minimum vertex cover	3.5 per edge
Set cover	3.9 per element
Assignment and scheduling	3.1 per row and column, 3.6 for precedence
Travelling salesman	3.1 twice: one city per position, one position per city
Max cut	none; constraint-free objective
Number partitioning	none; equality expressed directly by Consequence C
Portfolio with cardinality limit	3.2, optionally 3.9 for a budget
Minor embedding onto hardware	3.7 per chain

Derivations of the individual entries follow.

3.1 Selection of exactly one element (one-of- n encoding)

The constraint $\sum_{i=1}^n x_i = 1$ expresses that exactly one of n alternatives is chosen. It arises when encoding a categorical variable into binary variables, for example assigning a job to a single machine or a vertex to a single colour.

$$P = \left(\sum_i x_i - 1 \right)^2$$

Write $s = \sum_i x_i \in \{0, 1, \dots, n\}$. Then $P = (s-1)^2$, which is nonnegative and zero precisely for $s = 1$. The smallest nonzero value occurs at $s \in \{0, 2\}$ and equals $\Delta P_{\min} = 1$.

By Consequence C (with $a_i = 1$, $b = 1$):

$$P = 2 \sum_{i < j} x_i x_j - \sum_i x_i + 1$$

The matrix therefore receives $Q_{ii} = -\lambda$ and $Q_{ij} = 2\lambda$ for $i < j$, with the constant λ going into the separate offset.

Remark on structure: The term introduces a complete graph of couplings among all n variables. When embedding onto a sparse hardware topology, this is usually the most expensive part of the formulation.

3.2 Selection of exactly k elements

$$P = \left(\sum_i x_i - k \right)^2 = 2 \sum_{i < j} x_i x_j + (1 - 2k) \sum_i x_i + k^2$$

The expansion again follows Consequence C with $a_i = 1$, $b = k$. Compared with §3.1 only the linear coefficient and the constant change; the quadratic part is identical. $\Delta P_{\min} = 1$.

Typical occurrence: Cardinality constraints, where the size of the selected set is fixed in advance: a portfolio of exactly k assets, the placement of exactly k facilities, the choice of k representatives in a clustering problem.

3.3 Selection of at most one element

$$P = \sum_{i < j} x_i x_j$$

For $s = \sum_i x_i$, the number of pairs of selected variables is $\binom{s}{2} = \frac{s(s-1)}{2}$, hence $P = \frac{s(s-1)}{2}$. This is zero for $s \in \{0, 1\}$ and increasing for $s \geq 2$. The smallest nonzero value is $\Delta P_{\min} = 1$ at $s = 2$.

Choice against the slack alternative: The inequality $\sum_i x_i \leq 1$ could also be handled by §3.9, which would require one auxiliary variable. The form given here needs none and is therefore preferred. The general case $\sum_i x_i \leq k$ for $k \geq 2$ has no analogous slack-free form and must be handled by §3.9.

Typical occurrence: An exclusive choice that may also be declined: a machine that may remain idle, a time slot that need not be filled, an optional assignment. It differs from §3.1 exactly in permitting the empty selection.

The five pair patterns at a glance: Sections 3.4 to 3.8 all act on a single pair of variables and differ only in which of the four assignments they penalize.

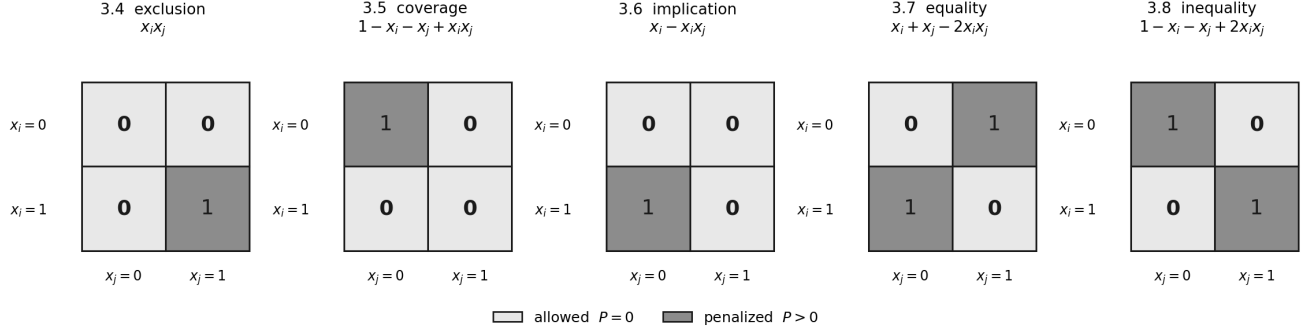


Figure 1: Each grid gives P for the four assignments of a pair. Light cells are the states the pattern permits, dark cells those it penalizes.

3.4 Mutual exclusion of a pair

$$P = x_i x_j$$

A special case of §3.3 for two variables. It corresponds to an edge in a conflict graph: the two variables must not be selected simultaneously. The term is zero in three of the four cases and equals 1 for $x_i = x_j = 1$.

Typical occurrence: Any pairwise incompatibility expressible as an edge of a conflict graph: the maximum independent set problem, jobs whose time intervals overlap, components that cannot be installed together. Applied to every edge of the graph.

3.5 Coverage by a pair

The constraint $x_i + x_j \geq 1$ requires at least one of the pair to be selected.

$$P = (1 - x_i)(1 - x_j) = 1 - x_i - x_j + x_i x_j$$

The derivation proceeds from the product of complements: the expression is nonzero precisely when both factors equal 1, that is, when $x_i = x_j = 0$. $\Delta P_{\min} = 1$.

Limits of applicability: For a general $\sum_{i \in S} x_i \geq 1$ with $|S| > 2$, the analogous product of complements $\prod_{i \in S} (1 - x_i)$ would have degree $|S|$ and would require quadratization by §3.10. It is more practical to convert to $-\sum_{i \in S} x_i \leq -1$ and handle it by §3.9.

Typical occurrence: Covering conditions in which each element must be served by at least one of two candidates: the minimum vertex cover problem, where each edge requires at least one of its endpoints to be selected.

3.6 Implication

Over $\{0, 1\}$ the constraint $x_i \leq x_j$ is equivalent to the implication $x_i = 1 \Rightarrow x_j = 1$. It is used for precedence relations and hierarchical dependencies.

$$P = x_i(1 - x_j) = x_i - x_i x_j$$

The expression is nonzero precisely in the single forbidden case $x_i = 1, x_j = 0$. $\Delta P_{\min} = 1$.

3.7 Equality of a pair

$$P = (x_i - x_j)^2 = x_i^2 - 2x_i x_j + x_j^2 = x_i + x_j - 2x_i x_j$$

The last equality is a direct application of idempotence (Consequence A) to both quadratic terms. Without it the expression would not be multilinear and could not be written into Q .

This pattern is used when embedding a logical variable into a chain of physical qubits, where it enforces their agreement. $\Delta P_{\min} = 1$.

Why chains arise: Hardware qubits have a fixed, small number of couplers. A logical variable coupled to more neighbours than that degree allows cannot sit on one qubit. It is therefore represented by several qubits tied together by this penalty, and the group behaves as one variable so long as the tie holds. A chain of length ℓ on a device of degree d carries at most $\ell(d - 2) + 2$ external couplings, so the chain length required grows with the degree of the variable in the formulation.

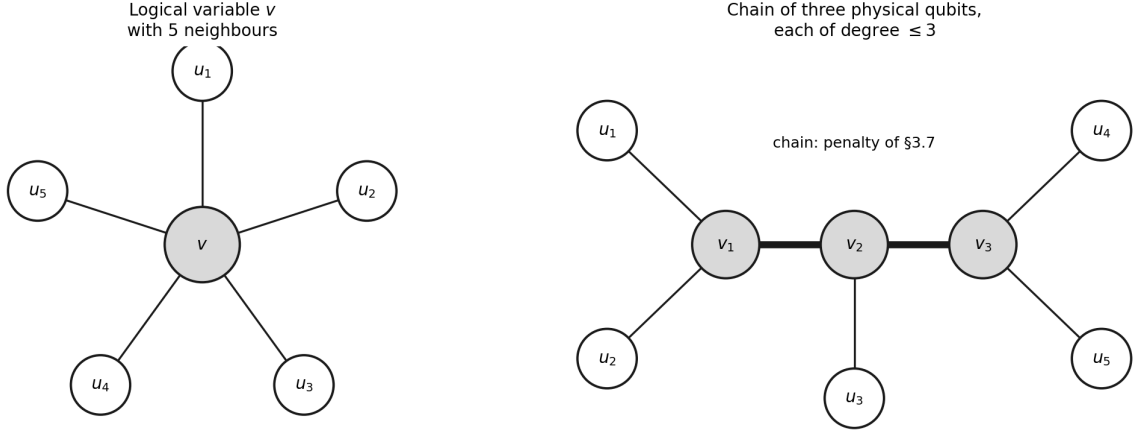


Figure 2: A variable with five neighbours cannot be placed on a single qubit of a degree-3 device. Split into a chain of three qubits held equal by the penalty of §3.7, it accommodates all five.

If the weight on the chain is too small the chain breaks, the qubits disagree, and the logical value is undefined. The fraction of broken chains is worth logging separately from the feasibility rate of §5.4, since the two fail for different reasons.

3.8 Inequality of a pair

$$P = (x_i + x_j - 1)^2 = 1 - x_i - x_j + 2x_i x_j$$

The expansion follows Consequence C with $a = (1, 1)$, $b = 1$. The term is zero precisely when exactly one of the two variables is selected. $\Delta P_{\min} = 1$.

Typical occurrence: Partition of a set into two classes, where a given pair must end up on opposite sides: max cut, balanced bipartitioning, parity conditions of exclusive disjunction.

3.9 Linear inequality with slack variables

The constraint $\sum_i a_i x_i \leq b$ is converted to an equality by introducing a nonnegative integer slack variable σ satisfying $\sum_i a_i x_i + \sigma = b$, which is then decomposed into binary variables s_k with coefficients c_k (see §4):

$$P = \left(\sum_i a_i x_i + \sum_k c_k s_k - b \right)^2$$

The expansion follows Consequence C over the concatenated coefficient vector $[a_1, \dots, a_n, c_1, \dots, c_m]$ and the concatenated variable vector $[x, s]$.

For the opposite direction $\sum_i a_i x_i \geq b$ the slack is subtracted: $P = (\sum_i a_i x_i - \sum_k c_k s_k - b)^2$.

The value of $\Delta P_{\min}$ depends on the coefficients a_i . For integer a_i and slack covering the entire feasible range, $\Delta P_{\min} = 1$. For non-integer a_i it must be determined numerically, or the coefficients rescaled appropriately.

Typical occurrence: Any capacity or budget limit: the knapsack problem, the capacity of a bin in bin packing, an upper bound on the consumption of a resource, or, in the opposite direction, a minimum required service level. This is the only pattern in the catalogue whose cost grows with the magnitude of b , since the number of slack variables is $\lceil \log_2(b+1) \rceil$.

3.10 Quadraticization of a product (Rosenberg substitution)

If the formulation contains a term of degree three or higher, that degree must be reduced to two. Introduce a new variable y replacing the product $x_1 x_2$ and add the enforcing term

$$P = 3y + x_1 x_2 - 2x_1 y - 2x_2 y$$

Verification by exhausting all eight combinations:

x_1	x_2	y	P	relation $y = x_1 x_2$
0	0	0	0	holds
0	0	1	3	violated
0	1	0	0	holds
0	1	1	1	violated
1	0	0	0	holds
1	0	1	1	violated
1	1	0	1	violated
1	1	1	0	holds

The term is nonnegative and zero precisely on the set satisfying $y = x_1 x_2$, so it meets the requirements of §1.2. $\Delta P_{\min} = 1$.

The substitution can be applied repeatedly; a term of degree d requires $d - 2$ auxiliary variables. The weight of this penalty is determined by the same procedure as for the other constraints.

4. Construction of slack variables

The slack σ from §3.9 takes values in $\{0, 1, \dots, b\}$ and is decomposed as $\sigma = \sum_k c_k s_k$ with binary s_k .

Requirement on the value range: The set of values representable by the chosen coefficients, $R = \{\sum_k c_k s_k : s \in \{0, 1\}^m\}$, must equal $\{0, \dots, b\}$. Each of the two inclusions has its own consequence when violated:

- $R \not\supseteq \{0, \dots, b\}$: some feasible solutions cannot be represented and are excluded from the search space. The QUBO optimum may then be worse than the true one.
- $R \supsetneq \{0, \dots, b\}$: the representable set is a proper superset of the feasible one. The formulation admits states that do not correspond to the original problem, and the penalty vanishes on them.

The direct binary expansion $[1, 2, 4, \dots, 2^{m-1}]$ satisfies the first inclusion but violates the second, since $\sum_k 2^k = 2^m - 1 \geq b$ with strict inequality for any b not of the form $2^m - 1$.

Correct construction: The last coefficient is truncated to the remainder:

$$c = [1, 2, 4, \dots, 2^{m-1}, b - (2^m - 1)], \quad m = \lfloor \log_2 b \rfloor$$

where the last element is omitted if it is zero. Then $\sum_k c_k = b$ and $R = \{0, \dots, b\}$.

```
def slack_coefficients(b: int) -> list[int]:
    coef, remainder, k = [], b, 0
    while remainder >= 2**k:
        coef.append(2**k)
        remainder -= 2**k
        k += 1
    if remainder > 0:
        coef.append(remainder)
    return coef
```

For $b = 11$ this gives $[1, 2, 4, 4]$ with sum 11. The direct expansion $[1, 2, 4, 8]$ sums to 15 and makes states 12 through 15 reachable, none of which corresponds to any solution of the original problem.

Check conditions: $\sum_k c_k = b$ and $m = \lceil \log_2(b + 1) \rceil$.

5. Choice of the penalty weight

5.1 Lower bound

Let f be the objective function and $\Delta f_{\max}$ the largest improvement in f obtainable by moving from a feasible solution to an infeasible one. For no infeasible solution to have lower energy than the optimum, it must hold that

$$\lambda > \frac{\Delta f_{\max}}{\Delta P_{\min}}$$

For the knapsack problem with values v_i and weights w_i , a conservative estimate is $\Delta f_{\max} \leq \max_i (v_i/w_i)$, so with $\Delta P_{\min} = 1$ this gives $\lambda > \max_i (v_i/w_i)$.

The bound is sufficient, not necessary; the smallest workable value of λ is usually lower and can be located by a sweep.

5.2 Upper bound

The upper bound is not algebraic but follows from the finite precision of the solver. As λ grows, $\max |Q|$ grows with it while the energy differences between feasible solutions remain unchanged. Their share of the total coefficient range shrinks until they fall below the resolving power of the hardware. The quantitative criterion is given in §7.

5.3 Recommended procedure

Start just above the lower bound from §5.1, add a margin of 10 to 20 percent, and run a sweep that tracks the fraction of feasible solutions and the precision requirement simultaneously. The optimal value is usually near the lower bound rather than an order of magnitude above it.

Constraints of differing scale: These require separate weights $\lambda_1, \lambda_2, \dots$. A single global weight leads to constraints with small coefficients being effectively unenforced, while constraints with large coefficients suppress the objective function.

5.4 The sweep in practice

1. Compute $\lambda_{\min}$ from §5.1.
2. Build a **geometric** grid, for instance $\lambda \in \{0.5, 1, 2, 4, 8, 16\} \times \lambda_{\min}$. The effect of λ is multiplicative, so an arithmetic grid wastes most of its points.

3. At each λ run $R \geq 50$ independent trials of a **classical** solver at fixed effort. Iterating on the quantum device is slower and adds measurement noise; save it for the final comparison of §9.
4. Record three quantities per λ : the feasibility rate $\rho(\lambda)$, the best feasible objective value, and $\beta(\lambda)$ from §7.2.
5. Take the **smallest** λ whose ρ meets the threshold you need; 0.9 is a common choice. Larger values buy nothing and raise β .
6. Confirm the chosen value with test 4 of §6.

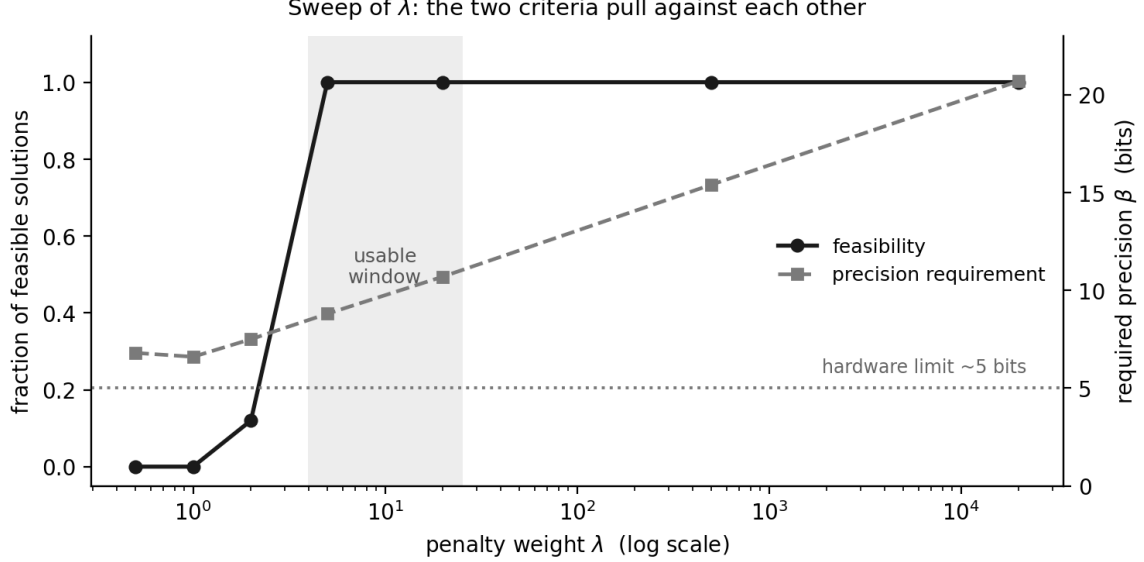


Figure 3: A sweep measured on the knapsack instance of §7.3. Feasibility saturates at $\lambda = 5$ while the precision requirement keeps climbing, so the usable window sits just above $\lambda_{\min} = 4$.

Detecting an over-large λ : Feasibility alone will not reveal it. In the same experiment $\lambda = 5$ and $\lambda = 20000$ both give full feasibility, yet the fraction of runs reaching the reference optimum falls from 0.74 to 0.41 and TTS rises from 68 to 174 milliseconds. Measure the fraction of runs reaching the **target quality**, not the fraction that are merely feasible.

Measurement	Reading	Action
$\rho \approx 0$, or the lowest-energy states are infeasible	λ too low	raise above $\lambda_{\min}$
$\rho = 1$, best objective stuck below the reference optimum	λ too high	lower toward $\lambda_{\min}$
$\rho = 1$, optimum reached, but p falls as λ grows	λ too high, less visibly	take the smallest λ with acceptable ρ
ρ never acceptable at any λ	formulation, not weight	return to §6, tests 1 and 3

5.5 Starting values by problem type

Every entry below follows from §5.1 and none is an independent rule. The table records $\Delta f_{\max}$ for standard problems, assuming $\Delta P_{\min} = 1$.

Problem	Objective	$\Delta f_{\max}$	Starting λ
Knapsack	$\max \sum_i v_i x_i$	$\max_i v_i / w_i$	$1.1 \times \Delta f_{\max}$
Maximum independent set	$\max \sum_i x_i$	1	2
Minimum vertex cover	$\min \sum_i x_i$	1	2
Travelling salesman	min tour length	$\max_{i,j} d_{ij}$	$1.1 \times \max_{i,j} d_{ij}$
Graph colouring	none, feasibility only	0	immaterial; see below
Max cut	max cut weight	—	no constraints, no λ

Problem	Objective	$\Delta f_{\max}$	Starting λ
Number partitioning	$\min(\sum_i a_i s_i)^2$	—	no constraints, no λ

Pure feasibility problems: Graph colouring has no objective to trade against the penalties, so λ only sets the overall scale of Q and cannot change the ranking of solutions. Fix it at 1 and spend the effort on the encoding instead.

Normalized weights: After normalizing the objective so that $\Delta f \approx 1$ as in §2.6, the starting values above collapse to $\lambda \approx 1.1$ across all constrained problems, which is what makes the normalization worth performing: one interpretable number replaces a table of problem-specific ones.

6. Validation suite

The tests are performed in the order given; the first failure halts the procedure.

#	Test	Property verified	Method
1	Slack value range	$R = \{0, \dots, b\}$	exhaustion of coefficient combinations
2	Encoding consistency	energy = f on feasible solutions	manual configurations, agreement to 10^{-9}
3	Agreement of optima	$\arg \min \text{QUBO} = \arg \min \text{original}$	exhaustion for $n \lesssim 22$ vs. CP-SAT, Gurobi
4	Penalty dominance	infeasible solutions have higher energy	random infeasible samples vs. optimum energy
5	Resolvability	hardware separates optimum from runner-up	$\max Q / (E_2 - E_1)$

Tests 1 through 4 verify **correctness of the translation**; test 5 verifies **solvability on the target hardware**. Satisfying the first group does not imply satisfying the second.

7. Precision criteria

7.1 An unsuitable metric

The dynamic range $\max |Q| / \min |Q|$ is invariant under scaling of the whole matrix. Multiplying Q by a constant leaves it unchanged even though the resolvability of solutions does change. As a standalone criterion it is therefore unusable.

7.2 A suitable metric

Let $E_1 < E_2$ be the energies of the best and second-best **feasible** solutions. The difference $E_2 - E_1$ is the quantity the solver must resolve in order to return the optimum rather than a suboptimal solution. The required precision in bits is

$$\beta = \log_2 \frac{\max |Q|}{E_2 - E_1}$$

Current annealing processors have an effective coupler precision of roughly 4 to 6 bits. If β lies substantially above this figure, the formulation cannot be transferred to the hardware regardless of its correctness.

7.3 Illustration

Measured on a knapsack instance with 7 items and capacity 11; the difference $E_2 - E_1$ remains equal to 1 and only $\max |Q|$ changes:

λ	Dynamic range (bits)	Requirement β (bits)
5	4.4	8.8
100	4.4	13.1
10000	4.4	19.7

The first column is constant and carries no information. The second shows that already at $\lambda = 5$ the requirement exceeds what the hardware can deliver.

8. Diagnostic table

Observation	Cause	Remedy
QUBO optimum is infeasible	λ below the bound of §5.1	raise λ above $\Delta f_{\max}/\Delta P_{\min}$
Solutions feasible but poor	λ too high, f suppressed	lower λ toward the bound
Energy disagrees with f	error in the construction of Q	test 2; check the diagonal (Consequence B) and the offset
Optimum differs only slightly	$R \supsetneq \{0, \dots, b\}$	test 1; truncate the last coefficient
Classical solver succeeds, hardware does not	insufficient precision	test 5; lower λ , rescale
Not even the classical solver succeeds	error in the formulation	return to test 3
Zero feasibility for all λ	constraints are contradictory	verify the original model
Energies differ by a factor of two	triangular and symmetric Q mixed	unify the convention of §0
Solver returns the worst solutions	maximization not negated (§2.2)	negate the objective, leave penalties positive
Optimum sits on the boundary of the value grid	discretization too coarse (§2.3)	refine the grid or widen its range
Single-flip search stalls between values	logarithmic encoding blocks local moves (§2.3)	switch to unary or domain-wall encoding

9. Comparing solvers

What is compared is not the best solution found but the expected time to reach a stated target quality with a given confidence:

$$\text{TTS} = t_{\text{read}} \cdot \frac{\log(1 - 0.99)}{\log(1 - p)}$$

where t_{read} is the time of a single run and p the fraction of runs that reached the target quality. The formula follows from requiring that the probability of at least one success in N independent runs reach 0.99, that is, $1 - (1 - p)^N \geq 0.99$.

Reference points, measured under an identical time budget:

1. An exact mixed-integer programming solver on the **original** formulation
2. A classical QUBO solver on the QUBO formulation (simulated annealing, tabu search)
3. A quantum solver on the QUBO formulation

Point 2 is the cheapest diagnostic available. If simulated annealing fails to find good solutions on a given formulation, the cause lies in the formulation and quantum hardware will not remove it.

10. Common errors

- Linear terms written off the diagonal, contradicting Consequence B
 - Omitted additive offset λb^2 when comparing energies against values of f
 - Mixing the upper-triangular and symmetric conventions for Q across functions
 - Slack with a non-truncated binary expansion (§4)
 - A single global weight λ for constraints of differing scale (§5.3)
 - Comparing energies across different values of λ ; they are not commensurable
 - A maximization left unnegated, or the negation applied to the penalty terms as well (§2.2)
 - Logarithmic encoding chosen for its low variable count without checking the precision requirement it produces (§2.3, §7.2)
 - A discretization grid so coarse that the grid optimum is far from the continuous one (§2.3)
 - A reference point consisting only of a weak heuristic rather than a good exact solver
-

11. Limits of applicability of QUBO

Conversion to QUBO is lossy in two respects. Slack variables and quadratization increase the dimension of the problem, and penalties replace explicit constraints with soft terms, thereby destroying structure that an integer programming solver would exploit to prune the search tree. On top of this comes the overhead of embedding onto the hardware topology, where a single logical variable consumes a chain of several physical qubits.

A good mixed-integer programming solver on the original formulation outperforms the QUBO route on the large majority of practical problems. This is not an argument against trying QUBO, but an argument for maintaining an honest reference point from the outset. Without one, the benefit of the quantum approach cannot be distinguished from the weakness of the baseline.